



Anàlisi i Disseny amb Patrons

Pràctica 2: Patrons de Disseny i d'Assignació de Responsabilitats

Enunciat

Per poder elaborar aquesta Pràctica partirem de la solució de la Pràctica 1.

A la Pràctica 1 vam aplicar alguns patrons d'anàlisi, el patró arquitectònic en capes i el patró Model Vista Controlador. A de més, vam dissenyar parcialment la capa de presentació per al cas d'ús *Make Reservation* (el controlador, la vista i algunes de les seves operacions). En aquesta segona part de la Pràctica, dissenyarem la capa de domini i la de serveis tècnics per al mateix cas d'ús (consulteu l'enunciat i la solució de la Pràctica 1, la descripció del cas d'ús, el diagrama de classes del MVC, etc ...).

Pregunta 1 [15%]

A la solució de la pregunta 4 de la Pràctica 1 es defineix l'operació `createReservation(...)` corresponent al cas d'ús *Make Reservation*, que resol la creació de reserves simples (no periòdiques) de forma anònima. Volem estendre el disseny inicial per a suportar, a més de les reserves simples anònimes, les reserves simples per part de clients identificats, complint en el possible el principi Obert-Tancat. Quin patró de disseny és adequat per a aquesta situació? Consulteu, en cas necessari, la documentació original dels patrons GoF o documentació complementària com

https://sourcemaking.com/design_patterns/factory_method i feu el diagrama de classes resultant de l'aplicació del patró de disseny escollit.

Resposta:

El patró de disseny adequat és el Mètode factoria (Factory Method). En la documentació original dels patrons GoF es contempla la variant d'operacions de creació parametritzades. En aquest cas utilitzem la presència el paràmetre opcional `dni` per determinar si la reserva és anònima o no. Es preserva el principi Obert-Tancat ja que per afegir nous tipus de reserva només cal afegir noves subclasses de factoria i no cal modificar codi existent.





Pregunta 2 [15%]

Definiu les responsabilitats de l'operació de creació de reserves simples per part de clients identificats definida a la pregunta anterior. Per definir les responsabilitats cal explicitar el següent:

- 1) Les condicions que s'han de complir just abans del inici de l'execució de l'operació (precondicions) i les comprovacions que aquesta operació ha de realitzar (comprovacions a aplicar a la pregunta 5).
- 2) Les conseqüències de les accions que fa l'operació en termes de creació / eliminació d'instàncies i associacions i d'assignació de valors a atributs (postcondicions).

Resposta:

Las responsabilitats de l'operació `createReservation(boxPoolCode: String, boxNumber: int, date: Date, timeBegin: Time, timeEnd: Time, dni: String):Reservation` són:

1) Precondicions:

- Box i BoxPool existeixen
- El client existeix

Comprovacions:

- El interval de temps a reservar no es solapa amb reserves existents del mateix box

2) Postcondicions:

- Es crea una instància `cr` de `CustomerReservation` assignant als seus atributs els valors rebuts com a paràmetres
- Es crea una associació entre `cr` i el box amb número `boxNumber` del `BoxPool` `boxPoolCode`
- Es crea una associació entre `cr` i el customer amb `dni` `dni`
- Es crea una instància `d` de `DateTimeRange`
- Es crea una associació entre `d` i `cr`

Pregunta 3 [10%]

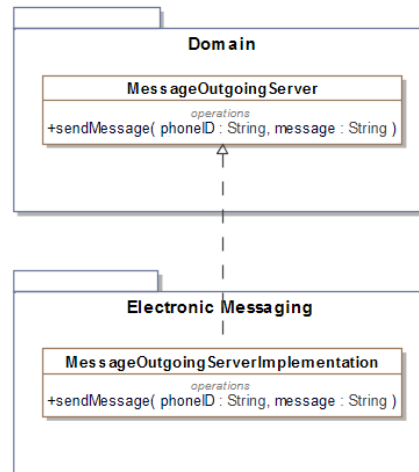
Ens indiquen que hi haurà almenys un requisit que implicarà l'enviament de missatges a telèfons mòbils pel que volem definir l'operació `sendmessage (phoneID: String, message: String)` És `Reservation` la classe idònia per implementar aquesta operació? Quins patrons de disseny són adequats en aquest cas? Dissenyeu el diagrama de classes del subsistema de missatgeria.

Resposta:

Segons el patró `Expert`, la mateixa `Reservation` és qui té la informació necessària per a fer-ho, però aquesta solució s'acobleria `Reservation` amb l'enviament de missatges, que és una responsabilitat ortogonal respecte a les que té fins ara. Per això, el patró `Fabricació` pura ens



suggereix que sigui una nova classe qui s'encarregui de la qüestió. Com que aquesta classe no s'implementarà a la capa de domini, aplicarem el patró Servidor abstracte i la convertirem en una interfície.



Pregunta 4 [15%]

La composició del text dels missatges a enviar ha de ser diferent segons el tipus de reserva ja que en el cas de ser una reserva de client, a més del codi de reserva, el PIN i el interval reservat, el missatge s'ha de dirigir de forma personalitzada amb nom i cognom. Per a això es preveu que la classe Reservation defineixi una operació `getComposedMessage()` que retorni un String amb el missatge compost. La possibilitat que cada subclasse implementi l'operació de forma específica preservaria el principi Obert-Tancat però té el inconvenient que podria passar que la jerarquia de Reservation acabés sent més experta en compondre missatges que en les pròpies reserves. Per resoldre el problema ens proposen utilitzar el **patró de disseny Estratègia** amb una operació ja implementada tal com

`composeMessage (desembre: Diccionari <String>): String`

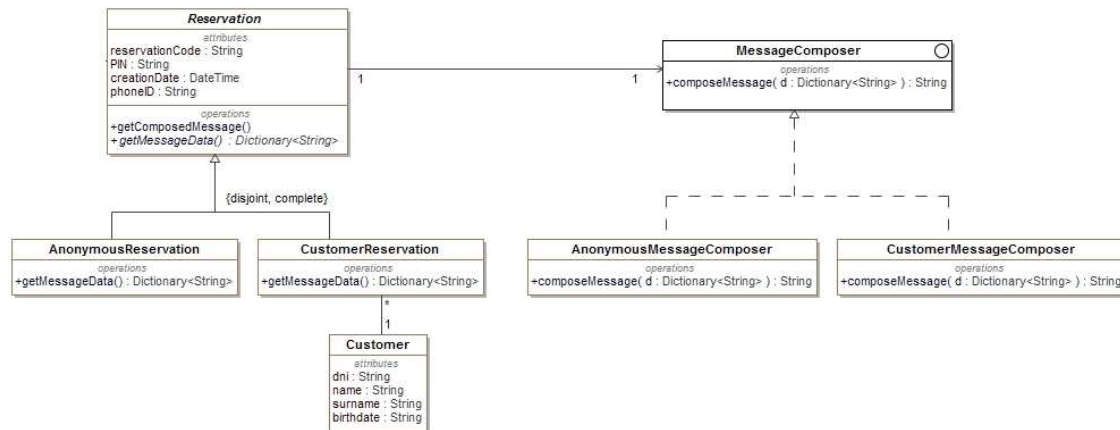
on dic conté les dades a mostrar en el missatge. Supposeu que disposem d'una operació ja implementada per confeccionar el diccionari amb les dades a incloure en el missatge tal com `Reservation :: getMessageData (): Diccionari <String>`.

Dissenyu el diagrama de classes, mostrant només les classes afectades per l'aplicació del patró Estratègia. Com s'hauria de modificar la resposta a la pregunta 2 per contemplar l'aplicació del mateix?

Resposta:



Una solució a aquest problema s'obté amb el patró Estratègia, delegant la lògica de composició a una jerarquia de classes en la qual cada subclasse implementa una estratègia de composició de text diferent.



Quan es crea una instància de **Reservation**, aquesta ha d'associar amb la subclasse corresponent de **MessageComposer** de manera que les reserves del tipus **AnonymousReservation** utilitzaran un **AnonymousMessageComposer** i les de **CustomerReservation** utilitzaran un **CustomerMessageComposer**.

A la resposta de la pregunta 2, que fa referència a les reserves de client, cal afegir les següents postcondicions:

- Es crea una instància cm de **CustomerMessageComposer**
- Es crea una associació entre cr i cm

Pregunta 5 [15%]

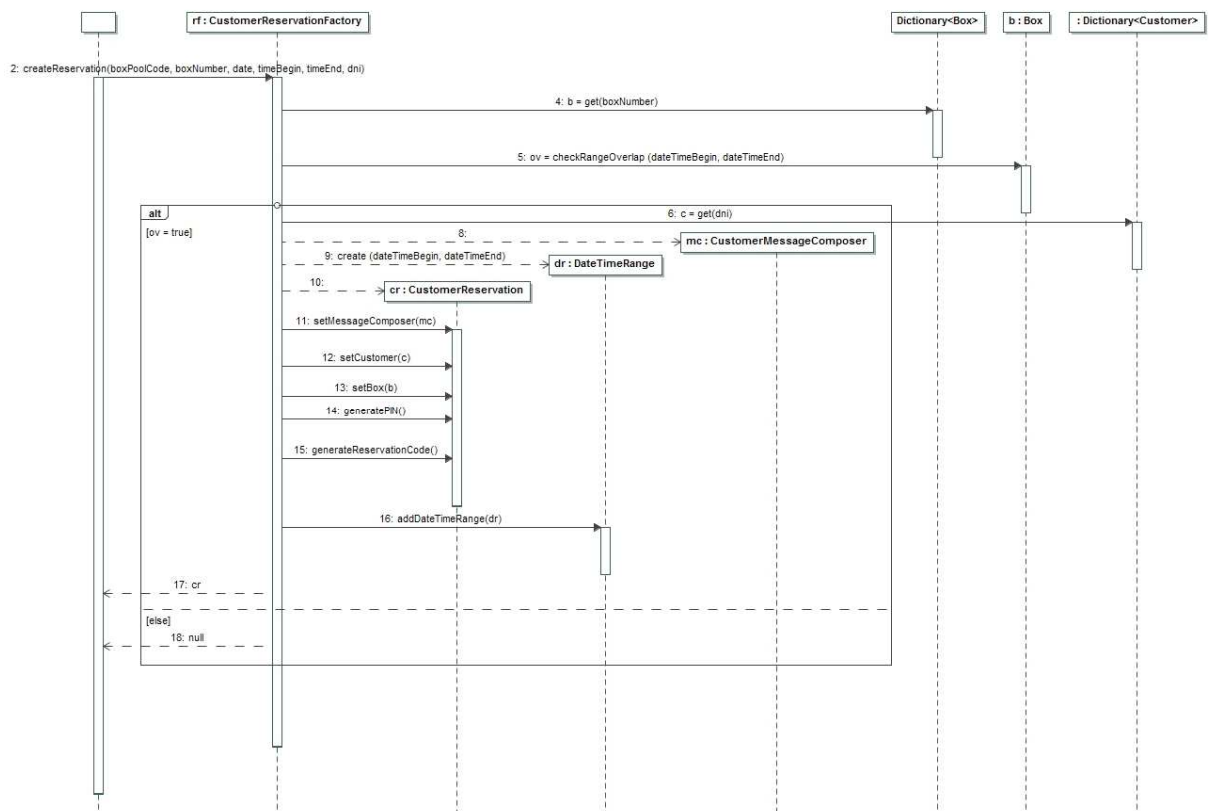
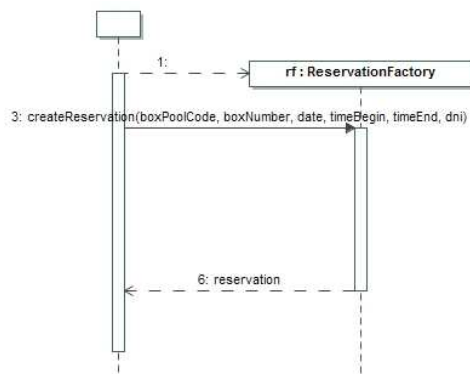
Partint de la vostra resposta a les preguntes anteriors, dissenyeu el diagrama de seqüència de l'operació de creació de reserves simples identificades (de client), indicant els principis de disseny aplicats. Supposeu que la classe **Box** disposa d'una operació que indica si un rang es solapa amb les reserves existents sobre un **Box** tal com

`checkRangeOverlap (dateTimeBegin: DateTime, dateTimeEnd: DateTime): Boolean`

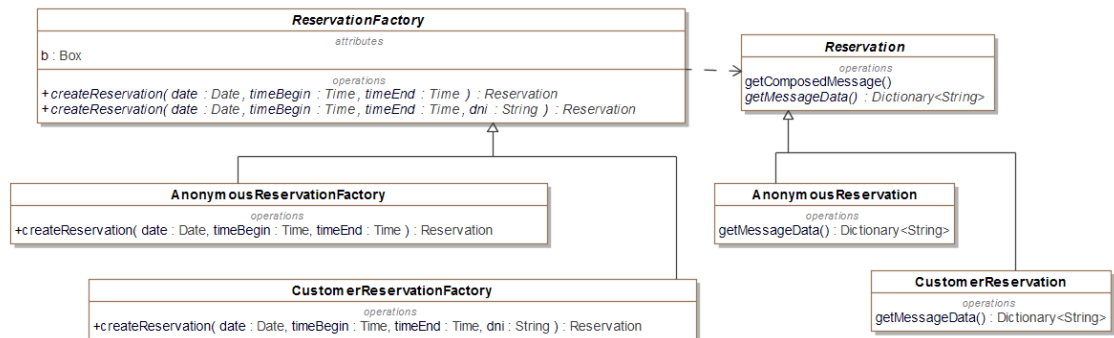
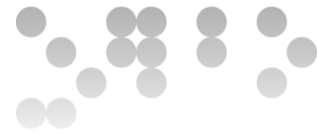
Suposeu també que **Reservation** té implementades les operacions `generatePIN ()` i `generateReservationCode ()`

En cas de no poder crear la reserva, el valor retornat serà null.

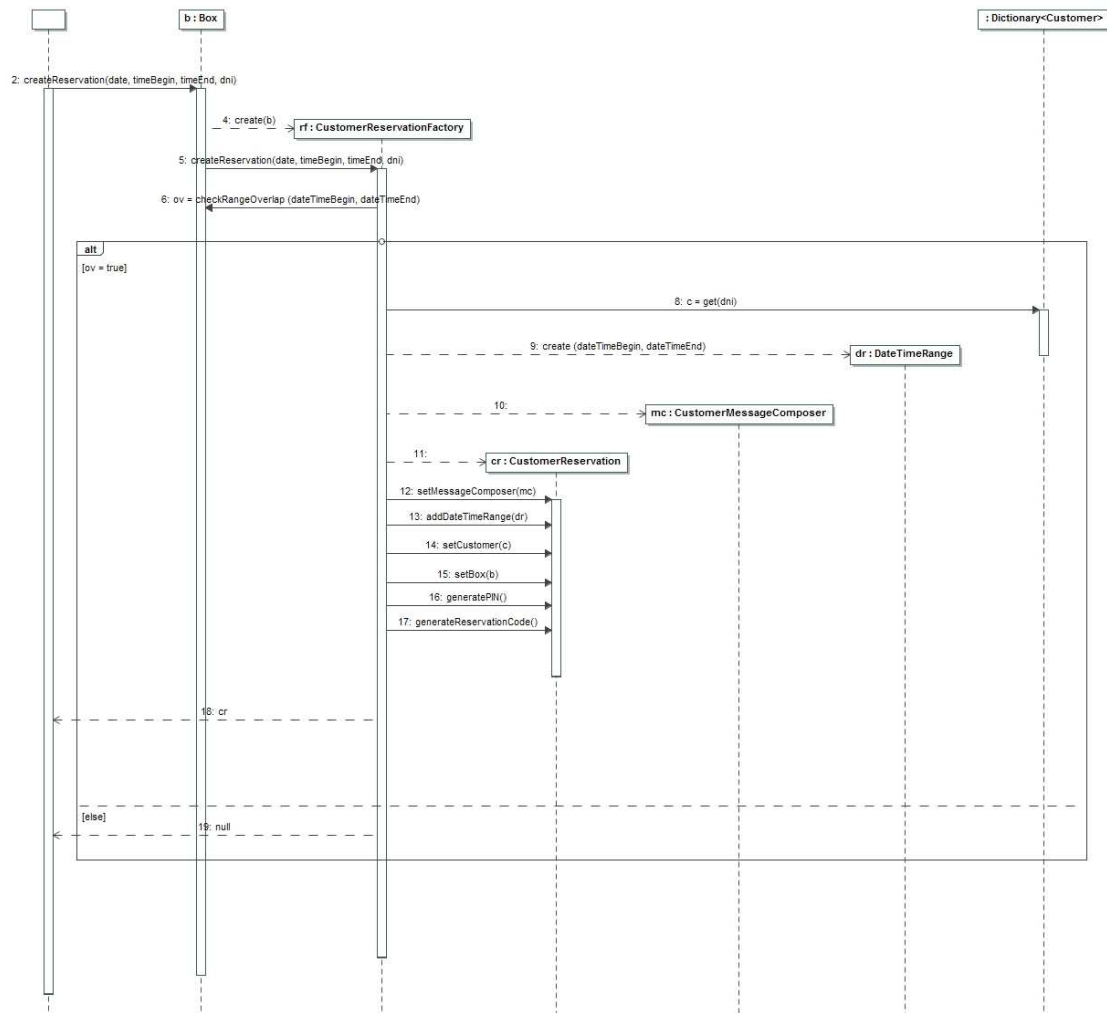
Resposta:



Una solució alternativa i també vàlida es pot obtenir combinant el patró Mètode Factoria amb el patró Creador, de forma que Box sigui responsable de crear reserves. El diagrama de classes es transformaria tal i com es mostra a continuació:



I el diagrama de seqüència quedaria tal com:



Aquest disseny compleix amb el principi de disseny de Baix acoblament, en el qual no cal utilitzar el diccionari de Box, mantenint l'ús del diccionari de Customer (els diccionaris són fruit de l'aplicació de patró Fabricació Pura).



Pregunta 6 [15%]

Ens demanen un servei consistent en enviar un missatge a tots els sol·licitants de reserves realitzades a partir d'una certa data. Dissenyeu el diagrama de seqüència d'una operació tal com:

BoxPool :: sendMessages (d: DateTime) i indiqueu els patrons de disseny utilitzats.

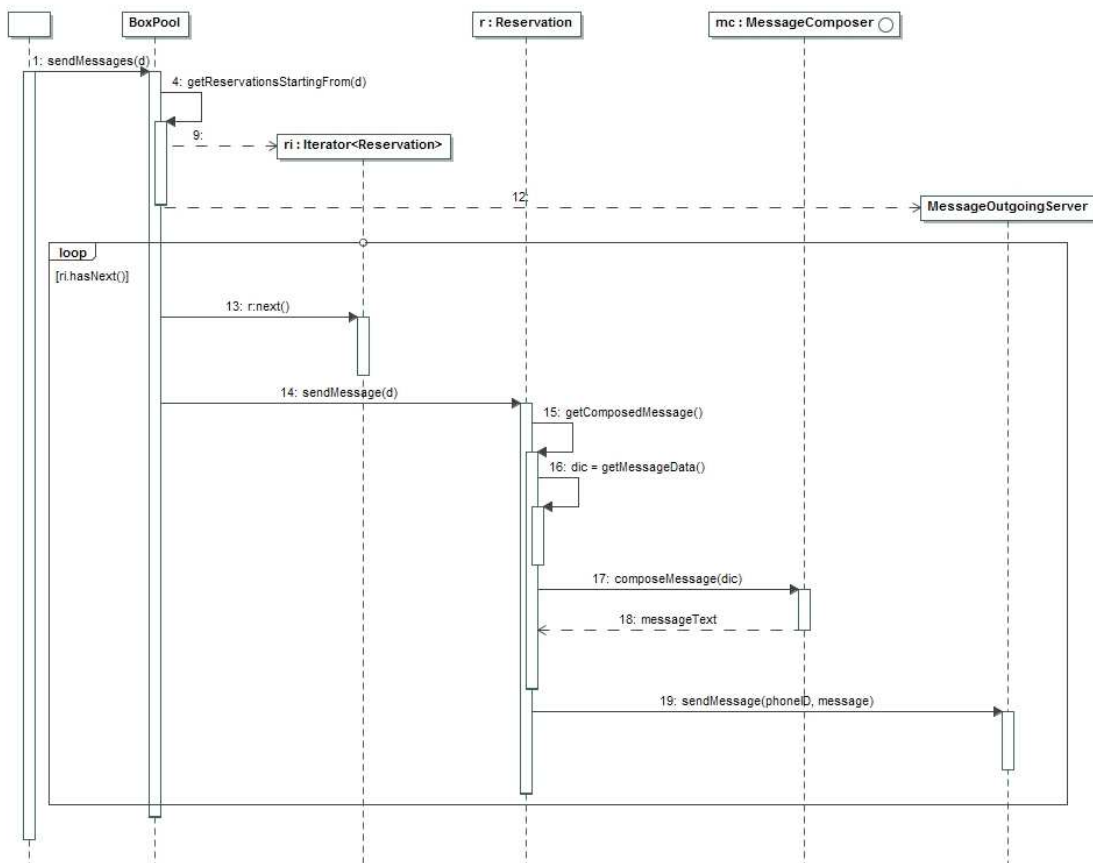
Suposeu que la classe BoxPool disposa d'una operació que retorna les reserves a partir d'una data / hora tal com

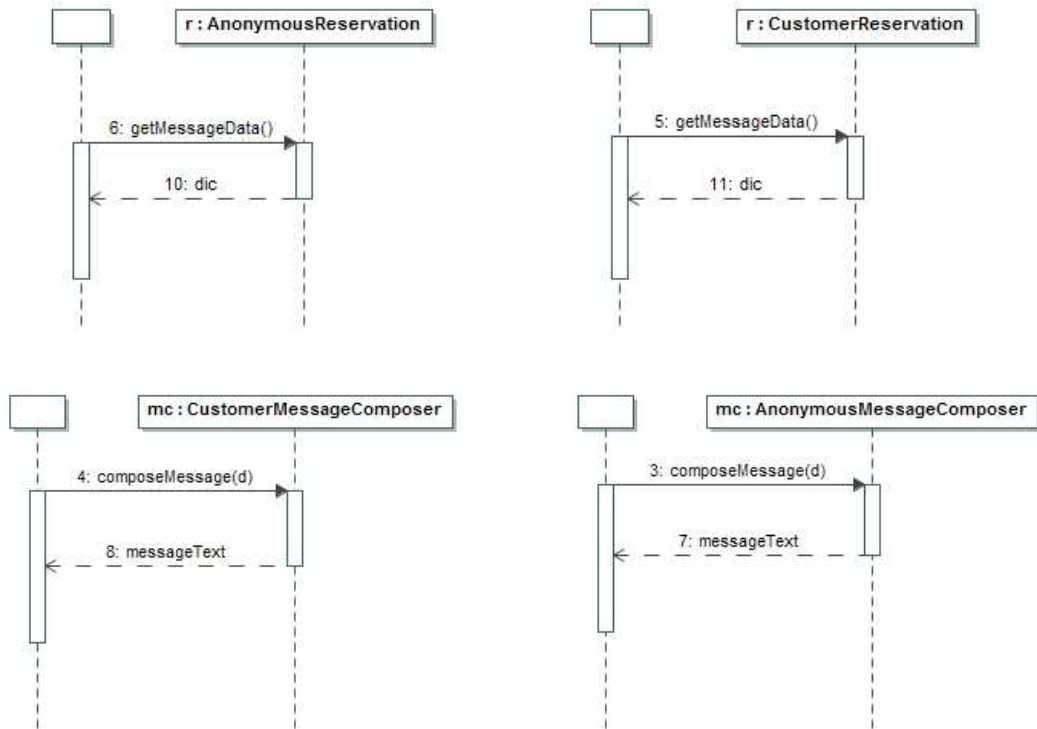
getReservationsStartingFrom (d: DateTime): Iterator <Reservation>

Utilitzeu els dissenys realitzats en les preguntes 3 i 4.

Resposta:

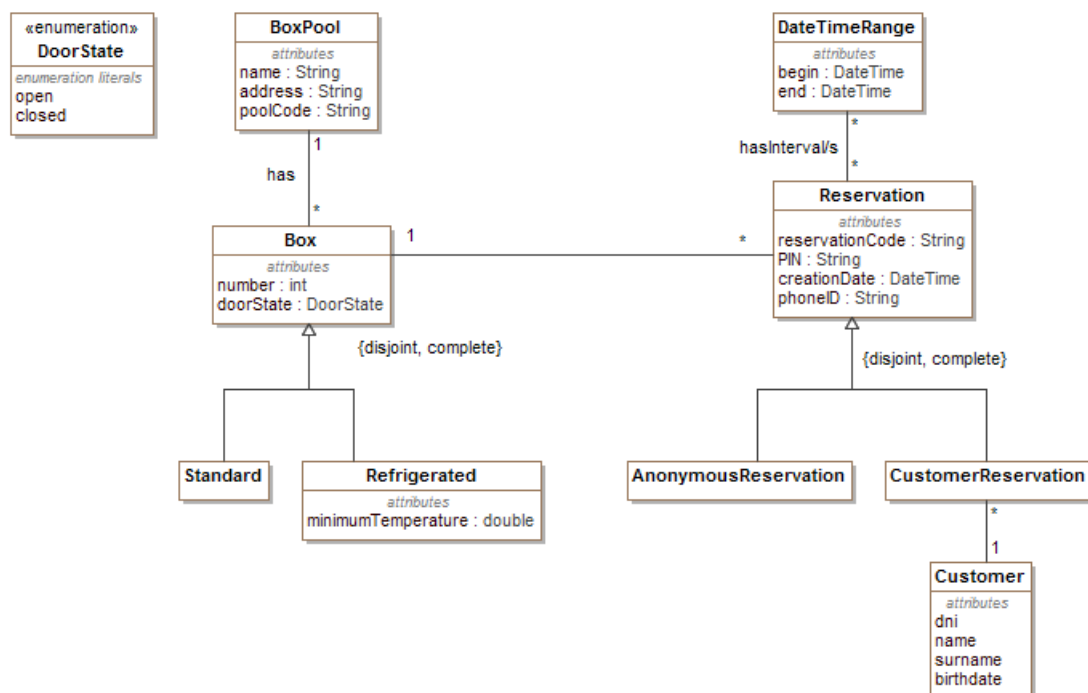
Aplicant el patró expert vam decidir que la classe Reservation serà responsable de l'operació sendmessage (d: DateTime). El patró Iterador ens proporciona un mecanisme per recórrer una col·lecció d'instàncies de Reservation de les que sabem que o bé són AnonymousReservation o bé CustomerReservation.

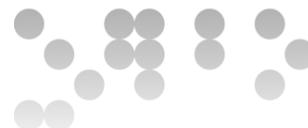




Pregunta 7 [15%]

Realitzeu el disseny lògic de la base de dades corresponent al diagrama estàtic de la Pràctica 1.





Resposta:

A continuació es mostra el disseny lògic de la base de dades. El subratllat indica claus primàries i la negreta els camps obligatoris (és el mateix conveni que es fa servir en els materials).

BoxPool (id, poolCode, name, address)

Box(id, idBoxPool, number)

{idBoxPool} es clave foránea a BoxPool(id)

BoxState(id, idBox, doorState, dateTime)

{idBox} es clave foránea a Box (id)

Refrigerated(id, minimumTemperature)

{id} es clave foránea a Box(id)

DateTimeRange(id, begin, end)

DateTimeRangeXReservation(id, idDateTimeRange, idReservation)

{idDateTimeRange} es clave foránea a DateTimeRange (id)

{idReservation} es clave foránea a Reservation (id)

Rerservation(id, idBox, reservationCode, PIN, creationDate, phoneID)

{idBox} es clave foránea a Box(id)

CustomerRerservation(id, idCustomer)

{id} es clave foránea a Reservation (id)

{idCustomer} es clave foránea a Customer(id)

Customer (id, dni, name, surname, birthdate)